The Journal of Open Source Software

quboify: Symbolic QUBO Modeling

Henri Karhula [1] **and Manuel Dahmen** [1]

1 Institute of Climate and Energy Systems, Energy Systems Engineering (ICE-1), Forschungszentrum Juelich GmbH, 52425 Jülich, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Vincent Knight

Submitted: 28 May 2026
Published: unpublished

Summary

`quboify` is an open-source Python package for constructing Quadratic Unconstrained Binary Optimization (QUBO) models from symbolic expressions while preserving symbolic parameters.

The primary use case of `quboify` is to take a `SymPy` model, and convert it into QUBO form while maintaining the symbolic expressions such that late-stage adjustments of parameters is possible. The qubo form uses data structures from `qubovert` which provides a reliable data layer with essential features for QUBO definition and editing.

This project originated as a fork of `mqt-qao`, but has since been rewritten almost entirely. The general modeling workflow and methods were adopted (Volpe et al., 2024b, 2024a), however the codebase has been rewritten to support preservation of symbolic expressions, modularity, and extensibility.

`quboify` has only limited links to existing quantum optimization solvers. As these solvers are expected to develop and change in the future, this toolkit offers an extendable solver-independent interface.

Statement of Need

QUBO formulations are widely used in combinatorial optimization and quantum computing. They serve as an intermediate representation for classical heuristics, Ising solvers, quantum annealers, and gate-based quantum algorithms such as QAOA.

It is common in research to find problem definitions in a `SymPy` model or other symbolic expression models. Therefore, the goal of `quboify` is to provide a toolkit that allows a streamlined QUBO modeling starting from such symbolic expressions while maintaining the option to adjust parameters up until the model is sent to a solver.

Existing packages, like `PyQUBO` or `mqt-qao` did not support this use case sufficiently. Nevertheless, `quboify` began as a fork of `mqt-qao`, adopting its high-level workflow for defining optimization problems: declaring binary variables, specifying objective functions, adding constraints, and transforming the problem into QUBO form.

State of the Field

Existing packages do aim to support similar use cases. `PyQUBO` supports the creation of QUBO formulations from mathemathical expressions and allows the late-stage modification of parameters, but lacks featurs for high-level modeling. `mqt-qao` supports the formulation of QUBO problems based on high-level concepts of variables, constraints and objective functions, but does not maintain the mathematical formulations such that parameter updates are supported. `qubovert` is a python package that offers great support in the creation and handling of QUBO formulations and modifications with custom data structures, but it lacks features

for extensive high-level modeling. `quboify` uses `qubovert` for its qubo handling features and it was originally forked from `mqt-qao` to make use of the established modeling approach and penalty weight optimization logic. The presented software is separated from `mqt-qao` because the core expression handling routines were entirely rewritten and the scope of the software was reduced to focus explicitly on the translation from optimization problem to qubo formulation and improvements of that workflow.

Software Design

While initially forked from `mqt-qao`, `quboify` made significant changes to the software architecture. Most importantly the expressions remain symbolic and the translation to the QUBO formulation was completely rewritten. In addition, the dependencies between the main classes `Problem`, `Constraints` and `Symbols` were simplified, and various classes (e.g., `Constraint`, `Variable`, `Parameter`) were introduced to improve readability and ease the development of future extensions.

When using the toolkit, one commonly follows these steps:

1. Define `Symbols` that will be used in the model.
2. Define `Constraints` based on the defined symbols.
3. Define `ObjectiveFunctions` based on the defined symbols.
4. Define a `Problem` from the `Symbols`, `Constraints`, and `ObjectiveFunctions`, which creates the symbolic QUBO form.
5. Create a `Solver` instance from the `Problem` and choose a strategy to handle the penalty lambdas.
6. Set parameters values using the `Problem.set_parameters` function.
7. Send the problem to a quantum optimization solver and receive the `Solutions`.
8. Inspect the `Solutions`.

The first three steps are specifically set up to allow almost seamless use of an existing `SymPy` model.

The most impactful feature of this toolkit is that it keeps an internal copy of the QUBO with symbolic parameters and only fills in numerical values before computations, which allows quick changes to parameter values without recreating the entire QUBO. This includes the handling of inequality constraints with parameters, where the encoding of these constraints require numerical bounds for slack variables.

Finally, some parts (such as definitions for specific problems and specific solver interfaces) were removed, to reduce the scope of the package.

Example:

The following example is a minor extension of the `mqt-qao` example. It now specifcally showcases the added support of potentially problematic variable names and expressions (see `a-0-0` and `1e-1`) which were interpreted incorrectly as subtractions by `mqt-qao`. It also showcases the use of parameter modification without reconstructing the QUBO (see parameter `p`) and also handles an inequality constraint with a parameter.

```python
from quboify import Constraints, ObjectiveFunctions, Problem, Solver, Symbols

variables = Symbols()
constraints = Constraints()
obj_func = ObjectiveFunctions()

a = variables.add_binary_variable("a-0-0")
b = variables.add_discrete_variable("b", [-1, 1, 3])
```

```
c = variables.add_continuous_variable("c", -2, 2, 0.25)
p = variables.add_parameter("p")

constraints.add_constraint(variables.encoding_constraints)
constraints.add_constraint("b + c*p >= 2", variable_precision=True)

obj_func.add_objective_function(a*p + b*c*p + c**2 + 1e-1)

problem = Problem(variables, constraints, obj_func)

solver = Solver(problem)

problem.set_parameters({"p": 2})
solutions = solver.solve_simulated_annealing()
print(f"Best solution: {solutions.best_solution}")
print(f"with objective function value: {solutions.best_solution_objective_values}")

problem.set_parameters({"p": -1})
solutions = solver.solve_simulated_annealing()
print(f"Best solution: {solutions.best_solution}")
print(f"with objective function value: {solutions.best_solution_objective_values}")
```

Research Impact Statement

By combining a familiar modeling workflow with a software architecture that maintains symbolic expressions, `quboify` offers a robust, flexible, and extensible toolkit for QUBO-based research. `quboify` enables easy non-expert usage and easy contributions by other developers, positioning the package as a maintainable modeling layer for combinatorial optimization and quantum computing applications.

The software is also actively being used in research within the "Quantum-based Energy Grids (QuGrids)" research project, and has also been used at the Institute of Climate and Energy Systems, Energy Systems Engineering (ICE-1), Forschungszentrum Juelich GmbH by colleagues (Hartmann et al., 2025).

Contributing

If you wish to contribute to `quboify` by creating Work Items (Issues) or Merge Requests (Pull Requests), you can login to the `quboify` Gitlab with your Google, Github or ORCID accounts using the "Helmholtz AAI" login option.

AI usage disclosure

No generative AI tools were used in the development of this software, the writing of this manuscript, or the preparation of supporting materials.

CRediT Statement

Henri Karhula: Conceptualization, Methodology, Software, Validation, Investigation, Writing - Original Draft, Visualization Manuel Dahmen: Funding acquisition, Supervision, Writing - Review & Editing

Acknowledgements

The project originated as a fork of `mqt-qao`. The authors acknowledge the original developers for establishing the modeling workflow and foundational concepts on which `quboify` builds.

The code and paper were written as part of the project "Quantum-based Energy Grids (QuGrids)", which is receiving funding from the programme "Profilbildung 2022", an initiative of the Ministry of Culture and Science of the State of North Rhine-Westphalia.

References

Hartmann, C., Rodellas-Gràcia, N., Wallisch, C., Pesch, T., Wilhelm, F. K., Witthaut, D., Stollenwerk, T., & Benigni, A. (2025). *Towards Quantum Algorithms for the Optimization of Spanning Trees: The Power Distribution Grids Use Case*. arXiv. https://doi.org/10.48550/ARXIV.2511.00582

Volpe, D., Quetschlich, N., Graziano, M., Turvani, G., & Wille, R. (2024a). A predictive approach for selecting the best quantum solver for an optimization problem. *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 1014–1025. https://doi.org/10.1109/QCE60285.2024.00121

Volpe, D., Quetschlich, N., Graziano, M., Turvani, G., & Wille, R. (2024b). Towards an automatic framework for solving optimization problems with quantum computers. *2024 IEEE International Conference on Quantum Software (QSW)*, 46–57. https://doi.org/10.1109/QSW62656.2024.00019